

Operating Systems

What They Are, Their Types, Examples, and Key Concepts

An **operating system (OS)** is the most important piece of software on any computer — it's the layer that makes the hardware usable and lets every other program run. This guide covers what an operating system actually does, the major types of operating systems, real-world examples, their core functions, and the advantages and disadvantages of different approaches.

1. What Is an Operating System?

An operating system is **system software** that manages a computer's hardware and provides common services for other software (applications) to run. It sits between the user, the applications, and the physical hardware, acting as an intermediary and a resource manager.

Without an OS, every application would need its own code to talk directly to the CPU, memory, storage, and peripherals — an enormously inefficient and error-prone approach. The OS handles that complexity once, so applications can simply request services through it.

2. Core Functions of an Operating System

2.1 Process Management

Creates, schedules, and terminates processes (running programs), deciding which process gets access to the CPU and for how long, so multiple programs can appear to run at once.

2.2 Memory Management

Allocates and frees RAM for running programs, keeps processes from interfering with each other's memory, and uses techniques like virtual memory to let systems run more than physically fits in RAM.

2.3 File System Management

Organizes how data is stored, named, retrieved, and protected on storage devices, presenting it to users and programs as a familiar structure of files and folders.

2.4 Device Management

Communicates with hardware peripherals (printers, keyboards, network cards, etc.) through device drivers, so applications don't need to know the technical details of each device.

2.5 User Interface

Provides a way for users to interact with the computer, whether through a graphical user interface (GUI) with windows and icons, or a command-line interface (CLI) using typed commands.

2.6 Security & Access Control

Enforces permissions, user accounts, and authentication, protecting the system and its data from unauthorized access or interference between users and programs.

2.7 Networking

Manages network connections and communication protocols, allowing the computer to send and receive data over local networks and the internet.

3. Types of Operating Systems

3.1 Desktop / Personal Computer OS

Designed for everyday use on personal computers and laptops, balancing usability, general-purpose application support, and a graphical interface.

Examples: Microsoft Windows, macOS, Linux distributions (Ubuntu, Fedora, Linux Mint).

3.2 Mobile Operating Systems

Built specifically for smartphones and tablets, optimized for touch input, battery efficiency, and app-store-based software distribution.

Examples: Android, Apple iOS, iPadOS.

3.3 Server Operating Systems

Designed to run continuously and reliably on servers, handling multiple simultaneous users and network services such as web hosting, databases, and file sharing.

Examples: Windows Server, Ubuntu Server, Red Hat Enterprise Linux (RHEL), CentOS.

3.4 Real-Time Operating Systems (RTOS)

Guarantees that tasks are completed within a strict, predictable time frame, which is critical for systems where delays could cause failure or danger, such as medical devices or industrial controls.

Examples: FreeRTOS, VxWorks, QNX.

3.5 Embedded Operating Systems

A lightweight OS built into a specific device (rather than a general-purpose computer), controlling just the functions that device needs.

Examples: Embedded Linux, Windows IoT, TinyOS (found in smart TVs, routers, appliances).

3.6 Mainframe Operating Systems

Runs on large-scale mainframe computers used by banks, governments, and large enterprises for high-volume, high-reliability transaction processing.

Examples: IBM z/OS, IBM z/VM.

3.7 Batch Operating Systems

Processes groups ("batches") of jobs with little or no user interaction during execution; jobs are collected, queued, and run sequentially. Common in early computing and some large-scale data processing today.

Examples: IBM's early OS/360 batch systems (historical); modern batch job schedulers within larger OSes.

3.8 Distributed Operating Systems

Manages a group of independent, networked computers so they appear to users as a single coherent system, sharing resources and processing power across machines.

Examples: Amoeba OS, Google's internal cluster systems (conceptually similar to distributed OS design).

4. Quick Reference Table

Type	Primary Use	Examples
Desktop OS	Personal computers, general use	Windows, macOS, Linux
Mobile OS	Smartphones, tablets	Android, iOS
Server OS	Hosting services, multi-user access	Windows Server, RHEL
Real-Time OS	Time-critical, predictable tasks	FreeRTOS, VxWorks, QNX
Embedded OS	Dedicated single-purpose devices	Embedded Linux, Windows IoT
Mainframe OS	Large-scale enterprise transactions	IBM z/OS
Batch OS	Sequential job processing	Job schedulers, legacy batch systems
Distributed OS	Networked computers as one system	Amoeba OS

5. A Closer Look at Major Desktop/Mobile OS Families

5.1 Microsoft Windows

The most widely used desktop OS worldwide, known for broad hardware and software compatibility, a large gaming library, and strong presence in business environments.

5.2 macOS

Apple's desktop OS, running exclusively on Mac hardware, known for tight hardware-software integration, design-focused tools, and a polished user interface.

5.3 Linux

An open-source, highly customizable OS available in many "distributions" (distros). Popular in servers, development environments, and among users who want full control over their system.

5.4 Android

Google's open-source mobile OS, based on the Linux kernel, running on the majority of the world's smartphones and known for its openness and hardware diversity across manufacturers.

5.5 iOS / iPadOS

Apple's mobile OS, known for a consistent, tightly controlled user experience, strong privacy features, and deep integration with other Apple devices and services.

6. Advantages & Disadvantages by Approach

6.1 Proprietary Operating Systems (e.g., Windows, macOS)

- **Advantages:** Polished user experience, strong official support, wide commercial software compatibility
- **Disadvantages:** Licensing costs, limited customization, less transparency into how the system works internally

6.2 Open-Source Operating Systems (e.g., Linux distributions)

- **Advantages:** Free to use, highly customizable, transparent source code, strong community support
- **Disadvantages:** Steeper learning curve, less out-of-the-box compatibility with some commercial software, support quality can vary by community/distro

6.3 Mobile Operating Systems

- **Advantages:** Optimized for touch and battery life, huge app ecosystems, frequent security updates
- **Disadvantages:** More restricted customization (especially iOS), app-store gatekeeping, hardware-specific limitations

6.4 Real-Time Operating Systems

- **Advantages:** Highly predictable timing, critical for safety and industrial systems
 - **Disadvantages:** Limited general-purpose functionality, often more complex to develop for
-

7. Summary

An operating system is the foundational software that manages a computer's hardware and enables every other program to run, handling process scheduling, memory, files, devices, security, and networking. Different situations call for different types of OS — desktop systems for everyday personal computing, mobile systems for phones and tablets, server systems for hosting services, and specialized real-time, embedded, or mainframe systems for dedicated or mission-critical tasks. Choosing between proprietary and open-source options, in particular, usually comes down to a trade-off between polish/support and cost/customization.